Pre-Read Documentation: JAX-RS for RESTful Web Services

This document serves as a comprehensive pre-read for understanding and working with **JAX-RS (Java API for RESTful Web Services)**. It covers the foundational concepts of RESTful services, introduces JAX-RS as a Java specification for building them, and outlines the key components you'll need to know, including dependencies, resource classes, endpoints, and deployment on Apache Tomcat.

**By reading this document, you will gain a foundational understanding of:**

- **RESTful Services:** Principles, benefits, and core concepts.
- **JAX-RS Specification:** What it is, its purpose, and its role in building RESTful APIs in Java.
- **Key JAX-RS Components:**
  - **Dependencies:** Libraries required to use JAX-RS.
  - **Resource Classes:** Java classes that define your API endpoints and logic.
  - **Endpoints:** How clients access your resources via URLs.
  - **HTTP Methods:** How standard HTTP verbs (GET, POST, PUT, DELETE) are used to interact with resources.
  - **Annotations:** How JAX-RS uses annotations to simplify development.
- **Apache Tomcat:** Introduction to Tomcat as a Servlet Container and how it's used to deploy JAX-RS applications.

**This document is structured as follows:**


1. **Introduction to RESTful Services**
   - 1.1 What are RESTful Services?
   - 1.2 Principles of RESTful Services
   - 1.3 Benefits of using RESTful Services
2. **Introduction to JAX-RS (Java API for RESTful Web Services)**
   - 2.1 What is JAX-RS?
   - 2.2 JAX-RS as a Specification
   - 2.3 Key Concepts in JAX-RS
3. **Core Components of JAX-RS Applications**
   - 3.1 Dependencies: Libraries and Frameworks

        - 3.1.1 JAX-RS API
        - 3.1.2 JAX-RS Implementation (e.g., Jersey, RESTEasy)
        - 3.1.3 Servlet Container (e.g., Apache Tomcat)
    - 3.2 Resource Classes: Defining API Logic
        - 3.2.1 What are Resource Classes?
        - 3.2.2 JAX-RS Annotations for Resource Classes
        - 3.2.3 Example Resource Class
    - 3.3 Endpoints and Paths: Exposing Resources via URLs
        - 3.3.1 What are Endpoints?
        - 3.3.2 The @Path Annotation
        - 3.3.3 Path Parameters and Query Parameters
    - 3.4 HTTP Methods and Operations
        - 3.4.1 Mapping HTTP Methods to Resource Methods
        - 3.4.2 Annotations for HTTP Methods (@GET, @POST, @PUT, @DELETE, etc.)
        - 3.4.3 Request and Response Handling
    - 3.5 Content Negotiation (Brief Introduction)
        - 3.5.1 @Produces and @Consumes Annotations
4. **Apache Tomcat and Deployment**
    - 4.1 Introduction to Apache Tomcat
    - 4.2 Tomcat as a Servlet Container
    - 4.3 Deploying JAX-RS Applications on Tomcat
        - 4.3.1 Packaging as a WAR File
        - 4.3.2 Configuration for JAX-RS in Tomcat (web.xml or annotations)
5. **Further Learning Resources**


**Let's dive into each section:**

---

# 1. Introduction to RESTful Services

## 1.1 What are RESTful Services?

RESTful services, or REST APIs (Representational State Transfer Application Programming

Interfaces), are a widely adopted architectural style for building web services. They provide a standardized way for different systems to communicate and exchange data over the internet, typically using the HTTP protocol.

Think of REST as a set of principles and guidelines that make web services:

- **Simple:** Easy to understand and use.
- **Scalable:** Can handle a large number of requests and users.
- **Flexible:** Can be used with various data formats and client technologies.
- **Interoperable:** Systems built in different languages and platforms can communicate effectively.

Instead of complex, proprietary protocols, REST leverages existing web standards like HTTP, URLs (URIs), and data formats like JSON and XML.

## 1.2 Principles of RESTful Services

REST is based on six key architectural constraints that define its characteristics:

1. **Client-Server:** Separation of concerns. Clients and servers are independent, allowing each to evolve separately. Clients are responsible for the user interface and user experience, while servers are responsible for data storage and processing.
2. **Stateless:** Each request from a client to a server must contain all the information necessary to understand the request. The server does not store any session state between requests. This makes services highly scalable and reliable.
3. **Cacheable:** Responses should be explicitly or implicitly labeled as cacheable or non-cacheable. Caching responses reduces server load and improves client performance.
4. **Layered System:** The architecture can be composed of hierarchical layers. A client may interact with an endpoint and not know whether it is directly connected to the end server or to an intermediary. This simplifies system complexity and promotes scalability.
5. **Code on Demand (Optional):** Servers may temporarily extend or customize the functionality of a client by transferring executable code (e.g., Java applets, JavaScript). While optional, it provides flexibility.
6. **Uniform Interface:** This is the most fundamental principle and is further broken down into four sub-constraints:

- **Resource Identification:** Resources are identified using URIs (Uniform Resource Identifiers). Clients interact with resources, not specific functions. For example, /customers/123 identifies a specific customer resource.
- **Resource Manipulation through Representations:** Clients manipulate resources by exchanging representations of those resources. Common representations include JSON and XML.
- **Self-Descriptive Messages:** Each message (request and response) contains enough information for the recipient to process it. For example, using standard HTTP methods and media types (Content-Type header).
- **Hypermedia as the Engine of Application State (HATEOAS):** (Advanced concept, less critical for initial understanding). Responses should contain links that allow clients to discover available actions and transition between application states dynamically.

### 1.3 Benefits of using RESTful Services

- **Simplicity and Ease of Use:** REST leverages familiar HTTP methods and URLs, making it easier to understand and use compared to complex protocols.
- **Scalability:** Stateless nature and caching mechanisms make REST services highly scalable.
- **Flexibility and Interoperability:** REST is not tied to any specific technology or platform. Services can be built in any language and consumed by clients built in any language.
- **Wide Adoption:** REST is the dominant architectural style for web APIs, with a large ecosystem of tools, libraries, and expertise.
- **Improved Performance:** Caching and efficient use of HTTP can lead to faster response times and reduced network traffic.
- **Modularity and Maintainability:** The client-server separation promotes modularity and makes it easier to maintain and evolve both client and server components independently.

## 2. Introduction to JAX-RS (Java API for RESTful Web Services)

### 2.1 What is JAX-RS?

**JAX-RS (Java API for RESTful Web Services)** is a Java specification that provides a standardized way to build RESTful web services in Java. It's part of the Java EE (Enterprise Edition) and now Jakarta EE platform.

JAX-RS **doesn't provide a concrete implementation** itself. Instead, it defines a set of **interfaces and annotations** that frameworks can implement to provide REST functionality.

### 2.2 JAX-RS as a Specification

Think of JAX-RS as a blueprint or a contract. It specifies *how* you should build RESTful services in Java, but it doesn't provide the actual building materials. This means you need to use a **JAX-RS implementation framework** to actually build and run your services.

**Popular JAX-RS Implementations:**

- **Jersey:** The reference implementation of JAX-RS, provided by Oracle (now Eclipse Foundation). It's widely used and feature-rich.
- **RESTEasy:** Another popular implementation, part of the JBoss/WildFly ecosystem.
- **Apache CXF:** A more general-purpose web services framework that also supports JAX-RS.

For learning purposes, **Jersey** is a good starting point due to its being the reference implementation and its comprehensive documentation.

### 2.3 Key Concepts in JAX-RS

JAX-RS simplifies RESTful service development in Java by using annotations and a component-based model. Key concepts include:

- **Resources:** Represent entities or data that the API exposes (e.g., customers, products, orders). In JAX-RS, resources are implemented as **Java classes**.
- **Resource Methods:** Operations that can be performed on resources (e.g., get a customer,

create a new order, update product information). In JAX-RS, resource methods are **methods within resource classes**.

- **Endpoints (Paths):** URLs that identify resources and operations. JAX-RS uses the **@Path annotation** to define the URL path for resources and methods.
- **HTTP Methods:** Standard HTTP verbs (GET, POST, PUT, DELETE) used to define the type of operation. JAX-RS uses annotations like **@GET, @POST, @PUT, @DELETE** to map HTTP methods to resource methods.
- **Request and Response Processing:** JAX-RS handles the mapping of HTTP requests to resource methods and the conversion of method return values to HTTP responses.
- **Content Negotiation:** JAX-RS supports handling different data formats (e.g., JSON, XML) based on client requests and server capabilities using annotations like **@Produces and @Consumes**.

---

## 3. Core Components of JAX-RS Applications

### 3.1 Dependencies: Libraries and Frameworks

To build a JAX-RS application, you'll need to include specific libraries in your project. Typically, using a build tool like Maven or Gradle will simplify dependency management. Here are the core categories:

#### 3.1.1 JAX-RS API

You need the **JAX-RS API** itself, which provides the core interfaces and annotations. In Maven, this dependency would look something like:

```
<dependency>
    <groupId>jakarta.ws.rs</groupId>
    <artifactId>jakarta.ws.rs-api</artifactId>
    <version>3.1.0</version> </dependency>
```

*(Note: With Jakarta EE, the package names have changed from javax.ws.rs to jakarta.ws.rs.*

*Use the appropriate version depending on your Jakarta EE/Java EE environment.)*

#### 3.1.2 JAX-RS Implementation (e.g., Jersey)

You need a concrete **JAX-RS implementation** to actually run your services. For Jersey, you would include dependencies like:

```
<dependency>
    <groupId>org.glassfish.jersey.containers</groupId>
    <artifactId>jersey-container-servlet</artifactId>
    <version>3.1.3</version> </dependency>

<dependency>
    <groupId>org.glassfish.jersey.inject</groupId>
    <artifactId>jersey-hk2</artifactId>
    <version>3.1.3</version> </dependency>

<dependency>
    <groupId>org.glassfish.jersey.media</groupId>
                      <artifactId>jersey-media-json-jackson</artifactId>
<version>3.1.3</version> </dependency>
```

- **jersey-container-servlet**: Provides the servlet container integration for deploying Jersey applications in servlet containers like Tomcat.
- **jersey-hk2**: Jersey's dependency injection framework.
- **jersey-media-json-jackson**: Support for handling JSON data using the Jackson library. You might need other media providers for XML or other formats.

You might need additional Jersey modules depending on the features you use (e.g., for multipart form data, validation, etc.). Refer to Jersey documentation for more details.

#### 3.1.3 Servlet Container (e.g., Apache Tomcat)

You need a **Servlet Container** to deploy and run your JAX-RS application. **Apache Tomcat** is a

popular open-source servlet container. You'll deploy your JAX-RS application as a web application (WAR file) to Tomcat.

## 3.2 Resource Classes: Defining API Logic

### 3.2.1 What are Resource Classes?

**Resource classes** are plain old Java objects (POJOs) annotated with JAX-RS annotations to define RESTful resources and their operations. They are the core building blocks of your API.

Each resource class typically represents a specific resource or a collection of resources.

### 3.2.2 JAX-RS Annotations for Resource Classes

Key annotations used in resource classes include:

- **@Path**: Class-level annotation to define the base URL path for the resource class.
- **@GET, @POST, @PUT, @DELETE, @HEAD, @OPTIONS, @PATCH**: Method-level annotations to map HTTP methods to specific methods within the resource class.
- **@Produces**: Class or method-level annotation to specify the MIME types (media types) that the resource method *produces* (sends back in the response).
- **@Consumes**: Class or method-level annotation to specify the MIME types that the resource method *consumes* (accepts in the request body).
- **@PathParam**: Parameter-level annotation to extract values from path segments in the URL.
- **@QueryParam**: Parameter-level annotation to extract values from query parameters in the URL.
- **@HeaderParam**: Parameter-level annotation to extract values from HTTP headers.
- **@CookieParam**: Parameter-level annotation to extract values from cookies.
- **@FormParam**: Parameter-level annotation to extract values from form parameters (in request bodies with application/x-www-form-urlencoded).
- **@MatrixParam**: Parameter-level annotation (less common) to extract values from matrix parameters in the URL.
- **@Context**: Parameter-level annotation to inject contextual objects like HttpServletRequest, HttpServletResponse, HttpHeaders, UriInfo, etc.

- **@BeanParam**: Parameter-level annotation to group multiple request parameters into a single Java Bean.

#### 3.2.3 Example Resource Class

```
package com.example.rest;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;

@Path("/customers") // Class-level @Path, base URL path for this resource
public class CustomerResource {

    @GET // Method-level @GET, maps HTTP GET requests to this method
    @Produces(MediaType.APPLICATION_JSON) // Produces JSON responses
    public Response getAllCustomers() {
        // Logic to fetch all customers from a database or service
        CustomerList customers = // ... fetch customers ...
        return Response.ok(customers).build();
    }

    @GET
       @Path("/{customerId}") // @Path at method-level, relative to class-level path
    @Produces(MediaType.APPLICATION_JSON)
        public Response getCustomerById(@PathParam("customerId") Long customerId) {
        // Logic to fetch a customer by ID
        Customer customer = // ... fetch customer by ID ...
        if (customer != null) {
            return Response.ok(customer).build();
```

```
        } else {
            return Response.status(Response.Status.NOT_FOUND).build();
        }
    }
    @POST // Method-level @POST, maps HTTP POST requests
      @Consumes(MediaType.APPLICATION_JSON)  //  Consumes  JSON  request
bodies
    @Produces(MediaType.APPLICATION_JSON)
    public Response createCustomer(Customer newCustomer) {
        // Logic to create a new customer
        Customer createdCustomer = // ... create customer ...
                                                                return
Response.status(Response.Status.CREATED).entity(createdCustomer).build
();
    }

    // ... other methods for PUT, DELETE, etc. ...
}
```

**Explanation of the Example:**

- **@Path("/customers")**: Any requests starting with /customers will be handled by this CustomerResource class.
- **@GET (no @Path on method)**: Handles GET /customers requests (returns all customers).
- **@GET @Path("/{customerId}")**: Handles GET /customers/{customerId} requests (returns a specific customer by ID). "{customerId}" is a path parameter.
- **@PathParam("customerId")**: Extracts the value of the customerId path parameter from the URL and injects it into the customerId method parameter.
- **@Produces(MediaType.APPLICATION_JSON)**: Indicates that the methods will return responses with Content-Type: application/json.
- **@POST**: Handles POST /customers requests (creates a new customer).
- **@Consumes(MediaType.APPLICATION_JSON)**: Indicates that the createCustomer method expects the request body to be in JSON format. JAX-RS will automatically convert

the JSON request body to a Customer object (if you have a suitable JSON provider like Jackson).

- **Response.ok(), Response.status(), Response.entity(), .build()**: Used to construct JAX-RS Response objects. Response is the standard way to return HTTP responses from JAX-RS resource methods.

## 3.3 Endpoints and Paths: Exposing Resources via URLs

#### 3.3.1 What are Endpoints?

In RESTful services, **endpoints** are the URLs that clients use to access your resources. They are essentially the public addresses of your API.

In JAX-RS, endpoints are defined using the @Path annotation.

#### 3.3.2 The @Path Annotation

- **Class-level @Path**: Defines the base path for all methods in the resource class.
- **Method-level @Path**: Defines a path relative to the class-level path for a specific method. If a method has no @Path, it inherits the class-level path.

**Examples:**

- @Path("/products") at class level.
  - @GET method (no @Path): Endpoint is /products (e.g., GET /products).
  - @GET @Path("/{productId}") method: Endpoint is /products/{productId} (e.g., GET /products/123).
  - @POST @Path("/search") method: Endpoint is /products/search (e.g., POST /products/search).

#### 3.3.3 Path Parameters and Query Parameters

- **Path Parameters**: Used to identify specific resources within a path segment. Defined using curly braces {} in @Path and extracted using @PathParam. Example: /customers/{customerId}.
- **Query Parameters**: Used to pass additional information to the server, often for filtering,

sorting, or pagination. Appended to the URL after a question mark ? and key-value pairs separated by &. Example: /customers?city=London&orderBy=name. Extracted using @QueryParam.

**Example:**

```
@GET
@Path("/items")
@Produces(MediaType.APPLICATION_JSON)
public Response getItems(
        @QueryParam("category") String category, // Extract query parameter "category"
    @QueryParam("minPrice") Double minPrice // Extract query parameter "minPrice"
) {
    // ... logic to fetch items based on category and minPrice ...
}
```

Request: GET /items?category=Electronics&minPrice=100

In this example, category will be "Electronics" and minPrice will be 100.

## 3.4 HTTP Methods and Operations

### 3.4.1 Mapping HTTP Methods to Resource Methods

RESTful services rely heavily on standard HTTP methods to indicate the intended action on a resource. JAX-RS annotations make it easy to map these HTTP methods to your resource methods.

**Common HTTP Methods and their typical RESTful usage:**

- **GET**: Retrieve a resource or a collection of resources. Should be **safe** (no side effects) and **idempotent** (making the same request multiple times should produce the same result).
- **POST**: Create a new resource. Typically used when the server is responsible for generating the resource ID.

- **PUT**: Update an existing resource or create a new resource at a specific URI if it doesn't exist. Should be **idempotent**.
- **DELETE**: Delete a resource. Should be **idempotent**.
- **PATCH**: Apply partial modifications to a resource.
- **HEAD**: Retrieve headers for a resource, without the response body. Useful for checking if a resource exists or getting metadata.
- **OPTIONS**: Retrieve the communication options available for a resource, such as allowed HTTP methods.

#### 3.4.2 Annotations for HTTP Methods

JAX-RS provides annotations to directly map these HTTP methods to your resource methods:

- **@GET**: For handling HTTP GET requests.
- **@POST**: For handling HTTP POST requests.
- **@PUT**: For handling HTTP PUT requests.
- **@DELETE**: For handling HTTP DELETE requests.
- **@PATCH**: For handling HTTP PATCH requests.
- **@HEAD**: For handling HTTP HEAD requests.
- **@OPTIONS**: For handling HTTP OPTIONS requests.

#### 3.4.3 Request and Response Handling

JAX-RS handles much of the boilerplate of request and response processing.

- **Request Processing:**
  - JAX-RS maps incoming HTTP requests based on the URL and HTTP method to the appropriate resource method.
  - It automatically extracts path parameters, query parameters, headers, and request body content based on annotations and method parameters.
  - It performs content negotiation to determine the request's media type (using Content-Type header) and deserializes the request body into Java objects (if @Consumes is specified).
- **Response Processing:**
  - The return value of your resource method is converted into an HTTP response.
  - JAX-RS performs content negotiation to determine the response's media type (based on

@Produces and client Accept header).
    - It serializes the Java object return value into the appropriate response format (e.g., JSON, XML).
    - It sets HTTP headers (like Content-Type, Status Code, etc.) in the response.
    - You can use the Response class for more fine-grained control over the response (setting status codes, headers, entity, etc.).

### 3.5 Content Negotiation (Brief Introduction)

**Content Negotiation** is the process of selecting the best representation (data format) for a resource when a client and server can handle multiple formats.

JAX-RS uses the following annotations for content negotiation:

- **@Produces**:
    - Specifies the MIME types (e.g., MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML, MediaType.TEXT_PLAIN) that a resource method *can produce* in its response.
    - Can be at the class level (applies to all methods in the class) or method level (for specific methods).
    - The server will look at the client's Accept header in the request to determine the preferred format. If there's a match with @Produces types, that format will be used in the response.
- **@Consumes**:
    - Specifies the MIME types that a resource method *can consume* in the request body.
    - Typically used with methods that handle request bodies (e.g., @POST, @PUT).
    - The server will look at the request's Content-Type header to ensure it matches one of the @Consumes types.

**Example:**

```
@GET
@Path("/data")
@Produces({MediaType.APPLICATION_JSON,  MediaType.APPLICATION_XML})  //
Produces both JSON and XML
```

```java
public DataObject getData() {
    // ... logic to fetch data ...
    return dataObject;
}
```

If a client sends a request with Accept: application/json, the response will be in JSON format. If the client sends Accept: application/xml, the response will be in XML format (assuming you have providers for both formats).

---

## 4. Apache Tomcat and Deployment

### 4.1 Introduction to Apache Tomcat

**Apache Tomcat** is a popular open-source web server and **Servlet Container**. It's primarily used to deploy and run Java Servlet and JSP (JavaServer Pages) based web applications.

Tomcat is lightweight, easy to configure, and widely used for deploying Java web applications, including JAX-RS applications.

### 4.2 Tomcat as a Servlet Container

Tomcat is specifically a **Servlet Container** (also known as a Web Container). This means it provides the runtime environment for Java Servlets and JSPs.

Key functions of a Servlet Container like Tomcat include:

- **Lifecycle Management of Servlets:** Tomcat manages the lifecycle of servlet classes (loading, instantiation, initialization, service, destruction).
- **URL Mapping:** Maps incoming URLs to specific servlets based on configuration.
- **Request and Response Handling:** Handles HTTP request and response processing, providing APIs for servlets to interact with requests and generate responses.
- **Security:** Provides security features like authentication and authorization.
- **Concurrency and Thread Management:** Manages threads to handle concurrent requests efficiently.

- **JSP Support:** Provides an environment for running JavaServer Pages (JSPs).

## 4.3 Deploying JAX-RS Applications on Tomcat

To deploy a JAX-RS application on Tomcat, you typically package it as a **WAR (Web Application Archive) file**.

#### 4.3.1 Packaging as a WAR File

A WAR file is a standard archive format for packaging Java web applications. It contains:

- **Servlet classes**, resource classes (in the case of JAX-RS), and other Java code (.class files).
- **JSP files**.
- **Static resources** (HTML, CSS, JavaScript, images).
- **web.xml deployment descriptor** (optional in modern servlet containers but can be used for configuration).
- **META-INF directory** (for metadata).
- **WEB-INF directory** (for configuration and libraries).
- **Libraries (JAR files)** your application depends on (typically placed in WEB-INF/lib).

Build tools like Maven and Gradle can easily package your JAX-RS application into a WAR file.

#### 4.3.2 Configuration for JAX-RS in Tomcat (web.xml or annotations)

To make Tomcat aware of your JAX-RS application and its resources, you need to configure a **JAX-RS application class** (often extending jakarta.ws.rs.core.Application or using @ApplicationPath annotation) and register it with Tomcat.

**Configuration Methods:**

1. Using web.xml (Deployment Descriptor):
   You can configure a servlet that acts as the JAX-RS application entry point in your web.xml file (located in WEB-INF directory). You typically register a servlet provided by your JAX-RS implementation (like Jersey's ServletContainer).

```
<web-app xmlns="jakarta.ee/xml/ns/jakartaee"
```

```
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
                    xsi:schemaLocation="jakarta.ee/xml/ns/jakartaee
jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
         version="5.0">

    <servlet>
        <servlet-name>JAX-RS Servlet</servlet-name>

<servlet-class>org.glassfish.jersey.servlet.ServletContainer</servl
et-class>
        <init-param>
            <param-name>jakarta.ws.rs.Application</param-name>

<param-value>com.example.rest.MyApplication</param-value>
</init-param>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>JAX-RS Servlet</servlet-name>
        <url-pattern>/api/*</url-pattern> </servlet-mapping>

</web-app>
```

- servlet-class: Specifies the JAX-RS servlet container (e.g., org.glassfish.jersey.servlet.ServletContainer for Jersey).
- init-param: Specifies the fully qualified name of your JAX-RS Application class (e.g., com.example.rest.MyApplication).
- servlet-mapping: Maps URLs (e.g., /api/*) to be handled by the JAX-RS servlet. This means requests starting with /api will be routed to your JAX-RS application.

2. Using @ApplicationPath Annotation (No web.xml Required - Servlet 3.0+):
   Modern Servlet containers (like Tomcat 7+ and later) and JAX-RS implementations support annotation-based configuration. You can use the @ApplicationPath annotation on your

JAX-RS Application subclass to define the base path for your API, and you may not need web.xml at all.

3.

```
package com.example.rest;

import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;
import java.util.HashSet;
import java.util.Set;

@ApplicationPath("/api") // Defines the base path for your JAX-RS application
public class MyApplication extends Application {

    @Override
    public Set<Class<?>> getClasses() {
        Set<Class<?>> resources = new HashSet<>();
            resources.add(CustomerResource.class); // Register your resource classes here
        // ... add other resource classes ...
        return resources;
    }
}
```

- @ApplicationPath("/api"): Defines that all endpoints in your application will be under the /api base path.
- getClasses() method: You need to override this method and return a Set containing all your resource classes that you want to expose in your API.

**Deployment Steps to Tomcat:**

1. **Build your JAX-RS application into a WAR file.**
2. **Start Apache Tomcat.**

3. **Deploy the WAR file to Tomcat.** You can do this by:
    - Copying the WAR file to Tomcat's webapps directory.
    - Using Tomcat's Manager web application to upload and deploy the WAR file.
    - Using Tomcat's command-line deployer tool.

Once deployed, your JAX-RS endpoints will be accessible at URLs starting with the configured base path (e.g., http://localhost:8080/your-application-name/api/...).

---

## 5. Further Learning Resources

To deepen your understanding of JAX-RS and RESTful services, here are some recommended resources:

- **Jersey Documentation:** eclipse-ee4j.github.io/jersey/ - Documentation for the Jersey JAX-RS implementation (Reference Implementation).
- **RESTEasy Documentation:** resteasy.dev/ - Documentation for the RESTEasy JAX-RS implementation.
- **RESTful Web Services by Leonard Richardson and Sam Ruby:** (Book) - A classic and comprehensive book on RESTful architecture.